UNIVERSIDAD DE LA DEFENSA NACIONAL

CENTRO REGIONAL UNIVERSITARIO CÓRDOBA - IUA

Facultad de Ingeniería

Ingeniería en Informática

Inteligencia Artificial

Lineamientos del Proyecto Final

Condiciones de Evaluación y Prerrequisitos

Docente: Ing. González, Hernando Alexis

2026

1. Contexto y Propósito del Proyecto

El Proyecto Final de la asignatura busca integrar los conocimientos teóricos y prácticos adquiridos durante la cursada mediante la resolución de un problema real.

Se espera que el alumno diseñe, entrene o implemente una arquitectura de Inteligencia Artificial que aporte una solución medible. El abanico de posibilidades es amplio y busca fomentar la creatividad. El proyecto debe enmarcarse en alguna (o la combinación) de las siguientes categorías tecnológicas:

- **Computer Vision (Visión Artificial):** Reconocimiento facial, biometría, sistemas de OCR (Reconocimiento Óptico de Caracteres) para digitalización de documentos, análisis de imágenes médicas, detección de anomalías visuales en procesos industriales o control de calidad.
- **Machine Learning Predictivo y Sistemas de Recomendación:** Pronóstico de series temporales (forecasting para finanzas, clima, o ventas), estimación de riesgos, algoritmos de recomendación de productos/contenido, o clasificación de datos estructurados.
- **Procesamiento de Lenguaje Natural (NLP) y LLMs:** Análisis de sentimientos en redes sociales, chatbots de atención al cliente, extracción automática de entidades en textos legales, resumen automático, o sistemas RAG (Generación Aumentada por Recuperación) sobre bases de conocimiento propias.
- **Data Engineering & IA:** Pipelines de extracción de datos (Web Scraping) acoplados en tiempo real a modelos de inferencia, limpieza automatizada de datasets complejos y detección de anomalías en grandes volúmenes de datos.
- **Procesamiento de Audio y Señales:** Sistemas de reconocimiento de voz (Speech-to-Text), síntesis de voz (Text-to-Speech), o clasificación de patrones acústicos y señales de sensores (IoT).
- **Aprendizaje por Refuerzos (RL):** Agentes autónomos para resolución de entornos de simulación, optimización de rutas logísticas o automatización de procesos de toma de decisiones.

2. Prerrequisitos Técnicos y Metodología de Trabajo

Para garantizar que el desarrollo sea estructurado, auditable y evaluable bajo estándares de la industria del software, los estudiantes deberán cumplir con los siguientes hitos antes de iniciar la etapa de codificación:

2.1. Gestión de Repositorio y Arquitectura de Software

El proyecto no puede ser un conjunto de scripts sueltos o un único *Notebook* monolítico. Se exige:

- **Estructura Base y Separación de Capas:** El repositorio debe separar claramente las responsabilidades. Se requiere un directorio para datos (`/data`, excluido del control de versiones), cuadernos exploratorios (`/notebooks`), código fuente modularizado (`/src` o `/api`), y artefactos generados (`/models`).
- **Control de Versiones:** Uso obligatorio de ramas para el desarrollo (ej. *Trunk-Based Development* o *GitFlow* simplificado). Los commits deben reflejar incrementos lógicos del sistema.

2.2. Propuesta Formal (Project Charter)

Como primer entregable, el equipo deberá asentar la propuesta técnica (puede ser el documento fundacional en el `README.md` o el *Issue #1* del tablero de gestión), detallando:

- **Definición del Problema:** Descripción funcional de la solución propuesta y modelado del entorno (PEAS, si aplica).
- **Origen y Naturaleza de los Datos:** Descripción del dataset a consumir (volumen estimado, variables principales, y consideraciones sobre posibles sesgos o limpieza necesaria).
- **Métricas de Éxito y Baseline:** Especificación de cómo se medirá objetivamente el éxito del modelo (ej. alcanzar un F1-Score $> 0{,}80$, reducir el tiempo de procesamiento, etc.) y cuál es el punto de comparación (baseline) actual sin emplear IA.

- **Límites del Alcance (Out of Scope):** Definición clara de qué funcionalidades **no** contemplará el sistema, para acotar el esfuerzo del MVP.

¿Qué entendemos por MVP?

Un **Producto Mínimo Viable (MVP)** no es un sistema programado a medias, ni un borrador plagado de errores. Es la versión más pequeña y estructuralmente sólida del sistema que permite validar la hipótesis principal del proyecto.

En el contexto de esta asignatura, el MVP debe demostrar exclusivamente que el modelo central de Inteligencia Artificial funciona, procesa los datos correctamente y devuelve un resultado útil (inferencia).

Ejemplo práctico:

- **Es un MVP (Enfoque correcto):** Una API sencilla (ej. usando FastAPI) o un *script* robusto de consola que recibe una imagen o un archivo de datos, lo pasa por el modelo ya entrenado, y devuelve una clasificación o predicción precisa (ej. un JSON con los resultados).
- **No es un MVP (Antipatrón):** Invertir las primeras semanas desarrollando un sistema de *login* con autenticación de usuarios, una base de datos relacional para historiales y un *frontend* complejo en React o Angular, dejando el entrenamiento y validación del modelo de IA para el último día.

El esfuerzo inicial de desarrollo debe concentrarse de forma excluyente en el *pipeline* de datos y en el "cerebro" del sistema (el modelo). Todas las interfaces administrativas, pantallas de usuario y módulos accesorios deben declararse explícitamente en los **Límites del Alcance (Out of Scope)** de esta primera iteración.

2.3. Entorno de Desarrollo y Reproducibilidad

En Inteligencia Artificial, un experimento que no puede ser reproducido por un tercero carece de validez técnica. Para asegurar la auditoría del proyecto, se exigen los siguientes

requisitos:

- **Aislamiento de Entorno:** Es obligatorio el uso de entornos virtuales (`venv`, `conda`) o contenedores (`Docker`). Se penalizará la entrega de proyectos anclados a instalaciones globales del sistema operativo.
- **Manifiesto de Dependencias:** El repositorio debe incluir un archivo de dependencias estático (como `requirements.txt`, `Pipfile` o `pyproject.toml`) orientado a Python ≥ 3.10. Es indispensable aplicar *version pinning* (fijar las versiones exactas de cada librería).
- **Semillas Aleatorias (Seeds):** Todo proceso estocástico, como la división del *dataset* (train/test) o la inicialización de pesos neuronales, debe utilizar una semilla global. Esto garantiza que la cátedra obtenga resultados determinísticos al validar el código.

3. Modalidad de Trabajo y Condiciones de Aprobación

La asignatura plantea dos caminos excluyentes para el desarrollo del proyecto. La elección de la modalidad impacta directamente en el alcance técnico exigido y en la condición académica final a la que puede aspirar el estudiante. No se permitirán cambios de modalidad una vez superada la primera instancia de revisión.

3.1. Camino 1: Modalidad Grupal (Tope: Promoción con Coloquio)

Orientado a fomentar el trabajo colaborativo y la integración de equipos. Al distribuirse la carga de desarrollo entre múltiples integrantes, el nivel de profundidad técnica individual es menor. Por este motivo, esta modalidad habilita como máximo a la obtención de la promoción con coloquio, o en su defecto, a la promoción con examen final.

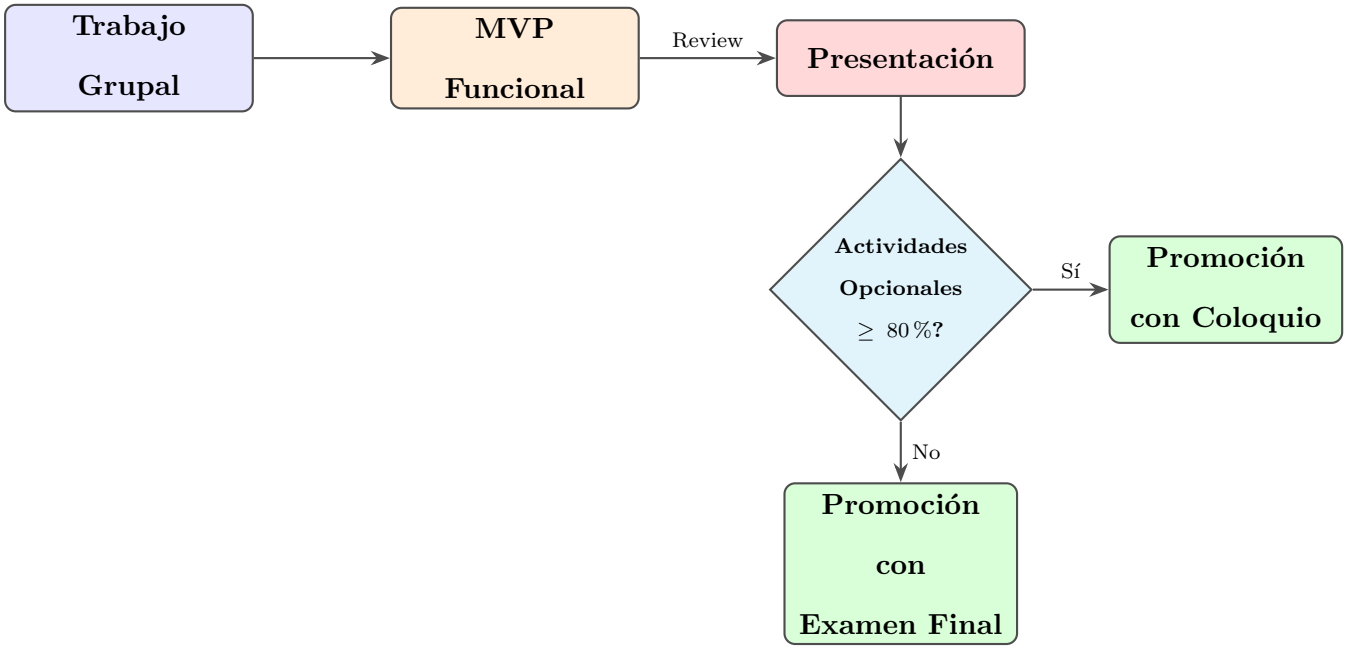
Fase de Desarrollo: Trabajo en equipo enfocado en la construcción rápida de un **MVP Funcional** (Producto Mínimo Viable).

Revisión (Review): Validación del código y de la viabilidad del modelo de IA con la cátedra.

Cierre: Defensa técnica y presentación del prototipo operativo.

Condición Académica: Evaluada según el rendimiento en la cursada:

- Promedio de Actividades Prácticas (Opcionales) $\geq 80\,\%$: **Promoción con coloquio**.
- Promedio de Actividades Prácticas (Opcionales) $< 80\,\%$: **Promoción con examen final**.



3.2. Camino 2: Modalidad Individual (Habilita Promoción Directa)

Orientado a estudiantes que busquen la máxima categoría de aprobación. Al no haber división de tareas, exige un dominio técnico integral (*end-to-end*) abarcando desde la ingeniería de datos hasta el despliegue del modelo.

Fase de Desarrollo Inicial: Trabajo autónomo hasta alcanzar el **MVP Funcional** de IA.

Revisión y Bifurcación: Tras la validación del MVP con el docente, se abren dos vías posibles según el alcance logrado:

- **No escala (Tope Promoción con Coloquio):** Si por limitantes de tiempo o complejidad técnica el estudiante decide no continuar el desarrollo, presenta únicamente el MVP.
- **Escalado a Proyecto Funcional:** El sistema se itera para sumar ingeniería de software y robustez alrededor del modelo. *(Ver detalle de requisitos más abajo)*. Esta es la única vía que habilita la promoción directa.

Cierre: Defensa técnica exhaustiva del sistema entregado.

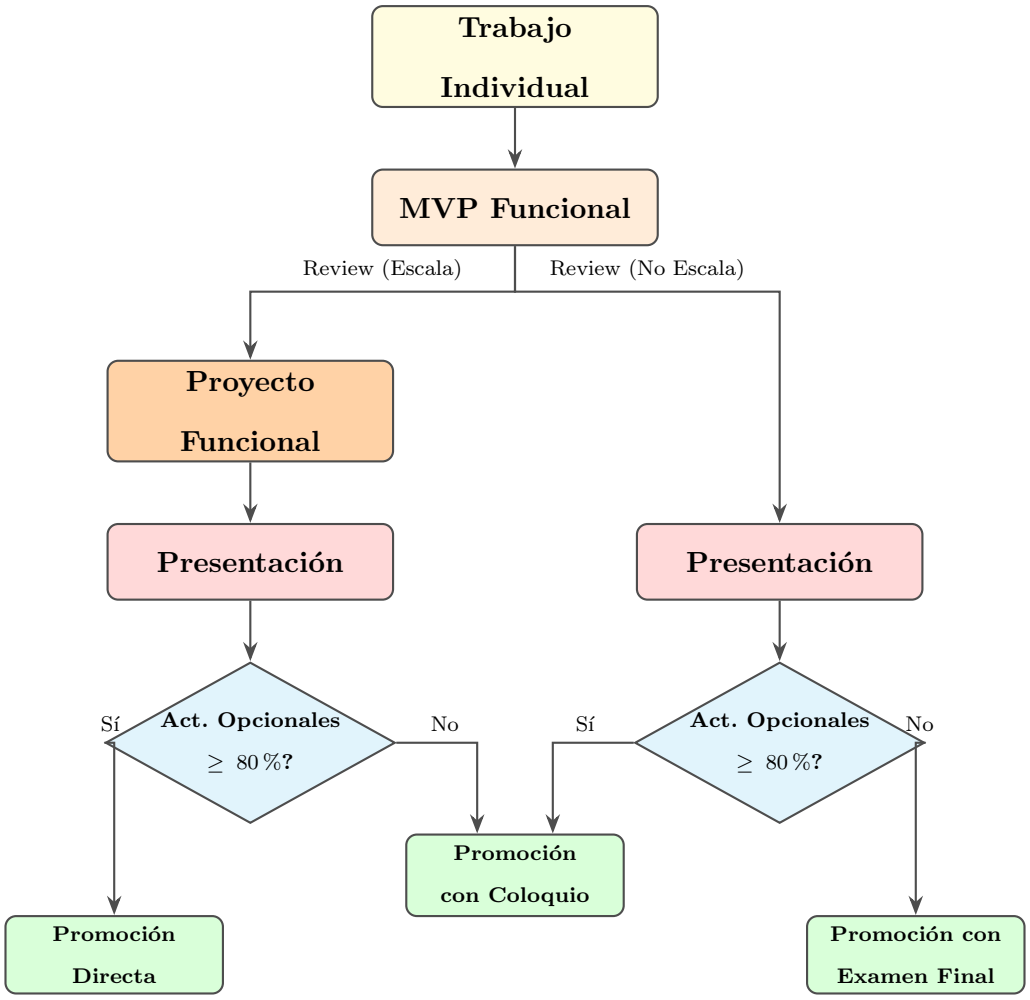
Condición Académica: Evaluada según el rendimiento en la cursada y la vía de desarrollo elegida:

- **Vía Proyecto Funcional (Escaló):**
  - Promedio Act. Opcionales $\geq 80\,\%$: **Promoción directa**.
  - Promedio Act. Opcionales $< 80\,\%$: **Promoción con coloquio**.
- **Vía MVP (No escaló):**
  - Promedio Act. Opcionales $\geq 80\,\%$: **Promoción con coloquio**.
  - Promedio Act. Opcionales $< 80\,\%$: **Promoción con examen final**.

Requisitos de Escalado: De MVP a Proyecto Funcional

Para que el sistema sea evaluado como un "Proyecto Funcional", no basta con que el modelo matemático infiera correctamente. El estudiante debe demostrar habilidades de Ingeniería de Software, implementando al menos **dos de los siguientes ejes**:

- **Arquitectura y Despliegue (API):** Envolver el modelo en una API REST documentada (ej. usando FastAPI). Incluir validación estricta de los datos de entrada (ej. esquemas Pydantic), respuestas HTTP adecuadas frente a errores de formato, y opcionalmente, *Dockerizar* la solución.
- **Interfaz y Usabilidad (GUI):** Construir una interfaz de usuario interactiva y amigable (ej. Streamlit, Gradio o framework frontend) que permita interactuar con el modelo sin tocar una sola línea de código, gestionando visualmente las excepciones y cargas de archivos.
- **Data Engineering e Integración:** Acoplar el modelo a un flujo de datos real. Por ejemplo, automatizar un proceso de *Web Scraping* para alimentar el modelo en tiempo real, o integrar una base de datos relacional (ej. MySQL) para persistir el historial de inferencias y analíticas.
- **MLOps y Calidad de Código:** Integrar un sistema de tests automatizados (`pytest`) para validar el procesamiento de los datos, sumar *logging* estructurado para auditoría de errores, o incluir herramientas de trazabilidad de experimentos (ej. MLflow).



4. Resumen Global de Condiciones Académicas

A modo de síntesis, la siguiente matriz consolida el impacto de la modalidad elegida, el alcance técnico entregado y el rendimiento durante el año sobre la condición académica final del estudiante. Constituye la regla definitiva de evaluación.

Matriz de Evaluación Final

Modalidad	Alcance del Desarrollo	Act. Opcionales	Condición Académica
Grupal	**MVP Funcional**	$< 80\,\%$	**Promoción con Examen Final**
	MVP Funcional	$\geq 80\,\%$	**Promoción con Coloquio**
Individual	**MVP Funcional** *(No escaló)*	$< 80\,\%$	**Promoción con Examen Final**
	MVP Funcional *(No escaló)*	$\geq 80\,\%$	**Promoción con Coloquio**
	Proyecto Funcional *(Escaló)*	$< 80\,\%$	**Promoción con Coloquio**
	Proyecto Funcional *(Escaló)*	$\geq 80\,\%$	**Promoción Directa**

5. Entregables y Buenas Prácticas

Independientemente de la modalidad elegida, el proyecto será evaluado bajo estándares profesionales de desarrollo de software:

- **Control de Versiones:** El avance debe reflejarse en un historial de Git lógico. Se evaluará el uso semántico de los mensajes de commit (ej. indicando el problema

resuelto y las horas invertidas).

- **Calidad del Código:** Estructura modular, uso de principios de arquitectura limpia y separación de responsabilidades (ej. desacoplar el procesamiento de datos del entrenamiento del modelo).
- **Documentación:** El repositorio debe contener un `README.md` exhaustivo que explique cómo levantar el entorno, ejecutar los scripts y reproducir los resultados obtenidos.

6. Rúbrica de Evaluación y Visibilidad

La evaluación prioriza el rigor científico y técnico en la resolución de problemas mediante Inteligencia Artificial. Si bien se exigen buenas prácticas de ingeniería de software para asegurar la robustez del sistema, el peso principal recae sobre la calidad del modelo y su validación estadística. Para los **Proyectos Funcionales**, es obligatorio publicar la documentación y resultados en `usuario.github.io`.

Rúbrica de Evaluación del Proyecto

Categoría	Criterio de Éxito	Peso
Implementación de IA	Selección de arquitectura, diseño del modelo, limpieza de datos y evitación de *data leakage*.	**45 %**
Validación y Métricas	Análisis estadístico riguroso, métricas (F1, RMSE, etc.) y reproducibilidad estricta con *seeds*.	**20 %**
Arquitectura y Código	Modularidad, limpieza de código y desacoplamiento de capas (Ingeniería de soporte).	**15 %**
Visibilidad (GitHub.io)	Calidad de la *landing page*, explicación clara de la arquitectura de IA y demo interactiva.	**10 %**
Git & MLOps	Trazabilidad de experimentos, historial de versiones y gestión de *issues*.	**10 %**

Nota: El componente de IA (Implementación + Validación) representa el **65 %** de la calificación final, reflejando el núcleo académico de la asignatura. El incumplimiento de la publicación en `github.io` para la modalidad individual (*Proyecto Funcional*) inhabilitará de forma automática la instancia de Promoción Directa.